Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

OPERATING SYSTEMS AND SYSTEMS PROGRAMMING (25CS2104E)

2026–27, Term-I

Project Problem Statement Submission Form

Section / Team Details

Section No.: 05

Team No.: 13

Project Title: System Call Demonstration and Monitoring System

Team Members

Roll Number	Student Name	Signature
2520030061	L. Ashmitha Reddy	
2520030208	A. Naga Jyothi	
2520030397	K. Harika	

Project Guide: Harika

1. Abstract

System Call Demonstration and Monitoring System is a Linux-based Operating Systems project that demonstrates how a C application interacts with the Linux kernel through system calls. Applications running in user space cannot directly access hardware or kernel resources; they must request services such as file access, process creation, program execution and synchronization through the system call interface. For students, this interaction is largely invisible. This project makes it visible by letting users submit their own C programs, run them in a controlled environment, and observe the actual system calls made during execution.

The project focuses on key Operating Systems concepts such as **user space and kernel space, the system call interface, file operations, process creation, program execution, process synchronization, and error handling**. The application compiles the submitted program with `gcc`, executes it in an isolated temporary directory, and captures real system call activity using `strace` with child-process tracing (`-f`). The raw trace is parsed into a clear table showing the system call name, process ID, arguments, return value, success or failure status, and timing.

The proposed solution is a working web-based application with a code editor/upload interface and a backend that compiles, runs and traces programs. It distinguishes source-level functions from the underlying syscalls (for example, `open()` commonly appears as `openat()`, `fork()` may appear as `clone()`, and `wait()` may appear as `wait4()`), and shows process trees built from actual PIDs. The expected outcome is a functional tool that provides practical understanding of how applications communicate with the Linux kernel, based on real execution data rather than fabricated examples.

2. Problem Statement

In an Operating System, user applications run in a restricted user space, while the kernel runs in a privileged kernel space. Every time an application needs to read a file, create a process, execute another program or wait for a child process, it must request the service from the kernel through a system call. Most beginners write programs using C library functions such as *printf()*, *fopen()* or *fork()* without realizing which system calls are actually invoked and how the kernel responds. Textbook diagrams explain the concept, but they do not show what really happens when a program runs.

This project addresses the problem of understanding system call behaviour by building a tool that compiles and runs user-submitted C programs in a controlled Linux environment and captures the actual system calls using `strace`. It does not rely on hard-coded demonstration programs or invented trace values; every system call, argument, return value, PID and timing shown is taken from the real execution of the uploaded program.

The project is relevant to Operating Systems and Systems Programming because it involves practical concepts such as the system call interface, user mode versus kernel mode, file descriptors, process creation and execution, process synchronization and error reporting through *errno*.

The project aims to show that system call tracing provides a simple and effective way to learn how application code interacts with the Linux kernel, while clearly separating facts observed through the trace from conceptual explanations of kernel behaviour.

3. Objectives

1. To demonstrate how system calls allow applications to request services from the Linux kernel.
2. To explain the distinction between user space and kernel space.
3. To accept C source code through file upload (.c) or an in-browser editor, with file type and size validation.
4. To compile submitted programs using GCC in an isolated temporary directory and report compiler errors and warnings.
5. To execute programs in a controlled environment and capture real system call activity using `strace`.
6. To display system call names, process IDs, arguments, return values, status and timing from the captured trace.
7. To explain the relationship between source-level functions and actual Linux syscalls (e.g., `open()` to `openat()`, `fork()` to `clone()`).
8. To demonstrate file operations, process creation, program execution, synchronization and error handling.
9. To present results through an interactive interface including a formal syscall table, detail view, process tree and execution overview.

4. Proposed Methodology

The project will be designed and implemented as a working application with a web front end and a Linux backend that compiles, executes and traces user-submitted C programs.

1. **Program Submission:** The user uploads a .c file or types code in the editor. The backend validates the file type and size and stores it in an isolated temporary working directory. Optional example programs are provided for testing.

2. **Compilation:** The program is compiled with `gcc -Wall -Wextra program.c -o program`. Compilation status, errors and warnings are shown. If compilation fails, the program is not executed.
3. **Controlled Execution:** The executable is run with a timeout and restricted resources inside the temporary directory. Program output, exit status, execution duration and timeout information are recorded.
4. **System Call Tracing:** The program is run under `strace -f` (with timestamps/durations where supported) to follow child processes. System call names, arguments, return values, errors and PIDs are captured, and the complete raw trace is preserved.
5. **Trace Parsing:** The raw trace is parsed, handling interrupted and resumed (unfinished) records correctly. Each event is stored with its sequence number, PID, syscall, arguments, return value, errno and duration.
6. **Source-to-Syscall Mapping:** Source-level functions found in the uploaded code are associated with observed syscalls only where a reliable association can be established. Events are labelled Direct or Underlying syscall; repeated calls are distinguished, and mappings are never invented.
7. **Visualization:** Results are shown in a formal syscall table (#, Source Line, Source Call, Actual Syscall, PID, Arguments, Return Value, Status, Time), a detail panel for each syscall, an execution overview, a process tree built from actual PIDs, and a conceptual user space / kernel space flow diagram.
8. **Testing:** The application is tested with programs for file I/O, fork/exec/wait, error cases (e.g., opening a missing file), compilation failures and infinite loops to verify correct compilation, tracing, parsing and display.

Execution Flow: Submit C Program → Validate → Compile (GCC) → Execute under strace → Parse Trace → Map Source to Syscalls → Display Results.

The methodology provides a practical demonstration of system calls, user/kernel interaction, process management and Linux tracing tools.

5. Operating Systems Concepts / Linux APIs Used

OS Concept / Linux API / System Call	**Purpose in the Project**
System Call Interface – user space to kernel space	Core concept demonstrated: applications request kernel services through system calls.
File Operations – `open()/openat()`, `read()`, `write()`, `close()`	Demonstrates file descriptors and file I/O, and how *open()* commonly appears as *openat()* in the trace.
Process Creation – `fork()` / `clone()`	Creates child processes; shows how the parent and child get different PIDs and return values.
Program Execution – `execlp()` / `execve()`	Demonstrates how the current process image is replaced by a new program.
Process Synchronization – `wait()` / `wait4()` / `waitpid()`	Shows the parent waiting for a child to finish.
Process Identification – `getpid()`, `getppid()`	Identifies processes and builds the process tree from actual PIDs.
Tracing – `strace -f`	Captures real system calls, arguments, return values, errors and timing, including child processes.

OS Concept / Linux API / System Call	Purpose in the Project
Error Handling – `errno`	Interprets failed syscalls (e.g., ENOENT) with error information.
Compilation – `gcc`	Compiles the uploaded C source into an executable.

6. Individual Contribution

Roll Number	Student Name	Individual Responsibility
2520030061	L. Ashmitha Reddy	Backend: file validation, GCC compilation, controlled execution with timeout, isolated working directory; testing.
2520030208	A. Naga Jyothi	Tracing and parsing: strace integration, trace parser, source-to-syscall mapping, execution statistics.
2520030397	K. Harika	Frontend: code editor/upload interface, syscall table, detail panel, process tree and kernel-flow visualization; documentation and example programs.

7. Tools / Platforms / Software Used

Tool / Platform / Software	Purpose
Linux / Ubuntu	Provides the operating environment for compiling, executing and tracing programs.
C Language / GCC	Language of the programs analysed; GCC compiles the submitted source code.
strace	Captures real system call activity of the executed program, including child processes.
Backend (Python / Node.js)	Handles uploads, compilation, execution, trace capture and parsing.
HTML / CSS / JavaScript	Builds the interactive interface (editor, tables, process tree, diagrams) in a rose, mauve and off-white theme.
Linux Terminal / Shell	Used to compile, run, test and monitor the application.
GitHub / Git	Version control and project submission.

8. Expected Outcome

The project is expected to produce a functional application that compiles, runs and traces user-submitted C programs and explains their interaction with the Linux kernel using real execution data.

1. Users can upload a .c file or enter code in an editor; invalid files are rejected with clear messages.
2. Programs are compiled with GCC; errors and warnings are displayed and failed builds are never executed.
3. Programs are executed in an isolated directory and traced with `strace -f`; the raw trace is preserved for inspection.
4. A formal syscall table shows sequence number, source line, source call, actual syscall, PID, arguments, return value, status and time, without flooding it with unrelated startup calls.
5. Selecting a syscall shows a detailed beginner-friendly explanation, including errno and duration, separating observed facts from conceptual explanations.

6. An execution overview shows compilation and execution status, program output, duration, PIDs, event counts and successful/failed syscall counts, all calculated from actual data.
7. A process tree built from actual PIDs explains fork, exec and wait behaviour, along with a conceptual user space / kernel space flow diagram.
8. Overall, the project provides practical understanding of system calls, user/kernel interaction and process management in Linux.

9. GitHub Repository

GitHub Repository Link:

Repository Created: [ ] Yes [ ] No

Faculty Name and Signature

Faculty Name: Harika

Signature:

Remarks:

Faculty Use

Project Approved: [ ] Yes [ ] No

Date:

Faculty Signature: